



OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

ELEKTRO- UND
INFORMATIONSTECHNIK

PRAKTIKUM INFORMATIK 1

M. Niemetz

Programmierwerkzeuge - Teil A

Oktober 2025, Version: 2.51

Fakultät Elektro- und Informationstechnik

1 Einleitung

In diesem Praktikum sollen Sie praktische Programmiererfahrung sammeln. Dabei geht es nicht primär darum, in der Vorlesung gesehene theoretische Konzepte *nachzumachen*, sondern vielmehr darum, selbständiges Programmieren zu *erlernen* um Sie in die Lage zu versetzen zu *gestalten*, wie es Aufgabe der Ingenieure ist.

Daher müssen Sie das Praktikum auch als eigenständige Lehrveranstaltung betrachten, in der Sie Dinge lernen, die sie in der Vorlesung nicht oder auch vielleicht erst später hören – auch wenn es natürlich eine enge Wechselwirkung zwischen den beiden Lehrveranstaltungen gibt, weil das Eine ohne das Andere nicht gut funktionieren kann.

Dies ist auch Bestandteil der akademischen Arbeitsweise, an die Sie sich nach und nach gewöhnen müssen: Nicht alles wird Ihnen haarklein und isoliert von allen anderen Inhalten vorgeführt, sondern die Inhalte sind vernetzt und Ihre Eigeninitiative beim Lernen neuer Themen wird zum unverzichtbaren Bestandteil Ihres Lernens.

Sie erhalten in diesem Praktikum mehrere solcher Anleitungen wie diese. Um Ihnen das Lesen zu erleichtern, sind die Arbeitsaufträge am Rand gekennzeichnet. Die Anzahl der Kaffeetassen soll Ihnen dabei einen Hinweis geben, wie umfangreich wir die Aktion einschätzen. Leider will das nicht viel heißen, hängt es doch davon ab, wie gut Sie vorbereitet sind und wie viele interessante Dinge Sie „am Rande“ der Bearbeitung des Arbeitsschritts entdecken.

Hinweis: Hinweise, die Ihnen bei der Bearbeitung helfen, werden hervorgehoben dargestellt. Sie sollen Ihnen helfen, die Aufgaben zu bewältigen.

Wichtig: Besonders wichtige Dinge, die Sie viel Zeit kosten können – oder Sie komplett in die Irre laufen lassen, wenn Sie sie nicht befolgen, werden als besonders wichtig dargestellt. Diese Dinge sollten Sie nicht ignorieren, wenn Ihnen an Ihrer Zeiteffizienz gelegen ist.

Interessante Hintergrundinformationen, die aber nicht zwingend zur Aufgabe gehören, sind manchmal in etwas hellerer Schrift eingefügt. Sie können diese Texte natürlich (ganz im Sinne der Freiheit des Studiums) überspringen. Aber natürlich schreiben wir nichts, von dem wir nicht denken, dass es Ihrem Kompetenzaufbau dienlich ist.



Lesen Sie diesen Abschnitt gründlich durch!

Hinweis: In der Prüfung IN1 fragen wir nicht ab, ob Sie was gelernt haben, sondern prüfen, ob Sie programmieren können.

Im Praktikum gibt es daher sogenannte „Abgaben“, in denen Sie mit Ihren Dozenten über Ihre Lösungen sprechen.

Dies verfolgt weniger den Zweck Sie zu prüfen (neu am Studium ist auch, dass Sie auch nichts lernen dürfen, wenn Sie das wollen) sondern dient dazu, dass Sie lernen, über Ihre Ideen und Programme mit anderen Entwicklern zu sprechen. Denn in der Praxis ist es so, dass Software-Entwicklung nicht-trivialer Programme wegen des großen Umfangs immer in Teams geleistet wird und daher die Kommunikation mit dem Team über das Arbeitsprodukt eine zentrale Fähigkeit im Berufsleben sein wird.

Hinweis: Sie werden noch feststellen, dass an der Hochschule häufig die wichtigsten Dinge (also die, wo Sie was lernen können) nicht verpflichtend sind. Abgesehen von den Praktika sind eigentlich nur die unwichtigen Dinge (da wo Sie nichts lernen) wie z.B. Prüfungen, verpflichtend.

Hinweis: Sie müssen jede Aufgabe in diesem Praktikum „abgeben“. Die Abgabe besteht dabei aus einem kurzen Gespräch über die Lösung mit Ihren Dozenten.

Damit Sie die Dinge, die Sie im Laufe dieser (und der folgenden) Aufgaben entdecken bei der jeweiligen Abgabe auch präsent haben, sollten Sie **unbedingt ein Praktikums-Tagebuch**¹ führen. Das ist natürlich nicht Pflicht (siehe oben). Aber Sie werden feststellen, dass Sie in der Hektik des Studiums Dinge sehr schnell vergessen. Da hilft es enorm, sich wichtige Erkenntnisse aufzuschreiben.

Führen Sie ein Praktikumsstagebuch!

2 Der Anfang

Sie beginnen in diesem Semester Ihren Einstieg in die Programmierung, indem Sie eine sogenannte „Hochsprache“ kennenlernen. Diese Sprachen zeichnen sich dadurch aus, dass sie weitgehend unabhängig von den Funktionen und Möglichkeiten der zugrundeliegenden Hardware eine konsistente Nutzerschnittstelle für unterschiedlichste Systeme bereitstellen.

Das Konzept besteht darin, dass Sie eine Idee haben, wie die Maschine arbeiten soll, dann der Maschine in dieser Hochsprache die zu erledigenden Aufgaben beschreiben, woraufhin dann Ihre

¹Profis nennen das auch ein Laborbuch.

Anweisungen durch ein Computerprogramm in für die Maschine ausführbare Anweisungen umgesetzt werden.

Dabei haben unterschiedliche Prozessoren natürlich unterschiedliche Fähigkeiten und damit auch Anweisungen – aber diese Komplikation nimmt ihnen das Computerprogramm (der sog. Compiler) ab.



Abbildung 1: Die Vorgehensweise beim Programmieren in Hochsprachen: Sie haben eine Idee, formulieren diese Ihre Idee in der maschinenunabhängigen Hochsprache und dann wird dieser Ablauf in maschinenspezifische Anweisungen für den Prozessor übersetzt. Diese Übersetzungsaufgabe übernimmt der Computer für Sie.

Zur Bearbeitung der in diesem Praktikum auf Sie zukommenden Programmieraufgaben benötigen Sie daher eine große Vielfalt entsprechender Werkzeuge wie Editor, Präprozessor, Compiler, Linker und Debugger.

Um den Umgang mit diesen Werkzeugen zu vereinfachen und eine möglichst konsistente Benutzerschnittstelle zu bieten, existieren sogenannte „Integrated Development Environments“ (IDE). Diese nehmen Ihnen sehr viel Arbeit ab und „verstecken“ die Vielfalt an Werkzeugen unter einer komfortablen Oberfläche.

Aber Sie wollen hier ja was lernen und daher beginnen wir zunächst damit, die wichtigsten Werkzeuge direkt zu verwenden um sie etwas kennen zu lernen, bevor wir sie dann aus Komfort- und Effizienzgründen wieder verstecken. Dadurch ist sichergestellt, dass Sie nachher beim Nutzen der IDE auch verstehen, was hinter den Kulissen abläuft und so die IDE auch effizienter nutzen können.

Wichtig: Bearbeiten Sie diese und die folgende Aufgabe der nächsten Woche möglichst gründlich und machen Sie sich mit Ihren Programmierwerkzeugen intensiv vertraut. Diese Investition wird nachher durch eine zügigere Bearbeitung der Programmieraufgaben belohnt.

3 Installation der notwendigen Werkzeuge

Damit Sie auf Ihrem privaten Rechner die Aufgaben dieses Praktikums bearbeiten können, benötigen Sie entsprechende Open-Source-Anwendungen, die Sie zuerst installieren müssen.

In den CIP-Pools der Hochschule entfällt natürlich die Installation.

Da die Programme, die Sie schreiben werden, als „einfache Textdateien“ gespeichert werden, ist ein Editor nötig, der mit solchen Dateien umgehen kann. Je nachdem welches Betriebssystem Sie verwenden, kann es sein, dass bereits entsprechende Editoren installiert sind. Auf manchen Systemen, müssen Sie das noch nachholen.

Linux: Hier ist vermutlich bereits einiges an Editoren installiert. Beispiele wären für die Textkonsole nano, pico, jpico, vim, emacs oder auch SciTE, gvim, jEdit für die graphische Oberfläche. Bei Bedarf können Sie entsprechende Editoren auch leicht über den Paketmanager nachinstallieren.

OS X:² Hier ist TextMate wohl ein sehr beliebter Editor.
<https://github.com/textmate/textmate>

Windows: Unter Windows ist in der Regel von Haus aus kein brauchbarer Texteditor verfügbar, Sie können allerdings behelfsweise mit notepad arbeiten.

Alternativ (dringend empfohlen!) können Sie das freie notepad++ unter

<https://notepad-plus-plus.org/>
verwenden, das auch in den Computerräumen (CIP-Pools) an der Hochschule verfügbar ist. Aber auch vim, SciTE, gvim, jEdit sind für Windows verfügbar.

Für die restlichen Werkzeuge ist es am Einfachsten, wenn Sie direkt die integrierte Entwicklungsumgebung CodeBlocks in der aktuellen Version (25.3) installieren.

Wichtig: Falls Sie **Windows** verwenden, achten Sie bitte unbedingt darauf, die Version **mit der Mingw-Toolkette** zu installieren. Sonst fehlen Ihnen nachher Compiler und weitere wichtige Werkzeuge.

Die Installationsdatei finden Sie auf der Seite von Code::Blocks (<http://www.codeblocks.org/downloads/binaries>).

Wichtig: (Nochmal!) Wählen Sie, falls Sie Windows als Betriebssystem nutzen, beim Download die Installationsdatei *mit* Compiler (mingw) aus, denn sonst müssen Sie den richtigen Compiler später selbst nachinstallieren. Vermeiden



Installieren Sie einen Texteditor



Installieren Sie die Entwicklungsumgebung CodeBlocks 25.03.

Sie ältere Versionen des Pakets, denn die darin enthaltene mingw-Version verhält sich an verschiedenen Stellen anders als hier im Praktikum beschrieben.

Hinweis: Falls Sie wissen was Sie tun, dürfen Sie natürlich auch gerne andere Entwicklungsumgebungen verwenden. Allerdings sollten Sie bedenken, dass das Praktikum derzeit auf die Verwendung von CodeBlocks optimiert ist, und Sie bezüglich der Adaptierung an Ihre spezielle IDE auf sich allein gestellt sind. Außerdem können Sie von den Dozenten keine Unterstützung für die Verwendung Ihrer individuell gewählten IDE erwarten – und auch Unterstützung bei der Fehlersuche funktioniert in der Regel nicht oder nur sehr eingeschränkt.

Falls Ihnen irgendwer eine andere Entwicklungsumgebung empfohlen hat, weil die „besser“ ist, bedenken Sie, dass Sie erst am Anfang stehen. Der Effizienzgewinn, den erfahrene Leute aus einer hochentwickelten IDE ziehen, stört bei diesem Einstieg eher.

Nach der Installation können Sie gleich mit den nächsten Punkt fortfahren.

4 Das erste Programm

4.1 Das Programm

Legen Sie ein Verzeichnis (manchmal werden die Verzeichnisse auch Ordner genannt) an, in dem Sie alle Dinge für das Praktikum PIN1 ablegen wollen.

Wichtig: Achten Sie *unbedingt* darauf, dass dieses Verzeichnis in seinem eigenen Namen kein *Leerzeichen*, keinen *Umlaut* und keine anderen Sonderzeichen besitzt und achten Sie auch darauf, dass alle übergeordneten Verzeichnisse ebenfalls diese Regel befolgen.

Erzeugen Sie dann in diesem PIN1-Verzeichnis ein weiteres Verzeichnis für diese erste einführende Aufgabe 0 und darin mit Hilfe Ihres Texteditors eine neue Datei. Diese soll die Dateiendung `.c` besitzen und darf im Dateinamen keine Umlaute, Sonderzeichen oder gar Leerzeichen enthalten. Geben Sie als Inhalt der Datei folgendes Programm ein.

Hinweis: Die Farben sind natürlich nicht wichtig, sie werden so, anders oder auch gar nicht von Ihrem Editor automatisch eingesetzt, um die Lesbarkeit Ihres Programms zu



Legen Sie Verzeichnisse an und erzeugen Sie Ihr erstes Programm.

verbessern. Für das Programm haben die Farben keine Bedeutung und werden auch nicht in der Datei gespeichert.

```
#include <stdio.h>
int main(int argc, char** argv)
{
    printf("Hello world!\n");
}
```

Speichern Sie die Datei ab. Damit haben Sie Ihr erstes C-Programm geschrieben.

4.2 Arbeiten mit dem Konsolenfenster

In diesem Abschnitt sollen Sie lernen, mit dem Computer über die Textkonsole anstatt mit der Maus zu interagieren.

Hinweis: Falls Sie bereits wissen, wie Sie eine Konsole öffnen und damit arbeiten können, ist dieser Abschnitt für Sie optional, Sie können ihn einfach überspringen.

Die Programme, mit denen wir im weiteren Verlauf arbeiten, haben keine mausgesteuerte graphische Benutzeroberfläche, weil sie ihre Arbeit ohne solche Interaktion verrichten können.

Um diese Programme ausführen und mit den nötigen Informationen versorgen zu können, benötigen Sie eine sogenannte Textkonsole, die ihrerseits wieder ein Programm ist, das auf Ihrem Rechner gestartet werden muss.

Wie das funktioniert, ist je nach System unterschiedlich:

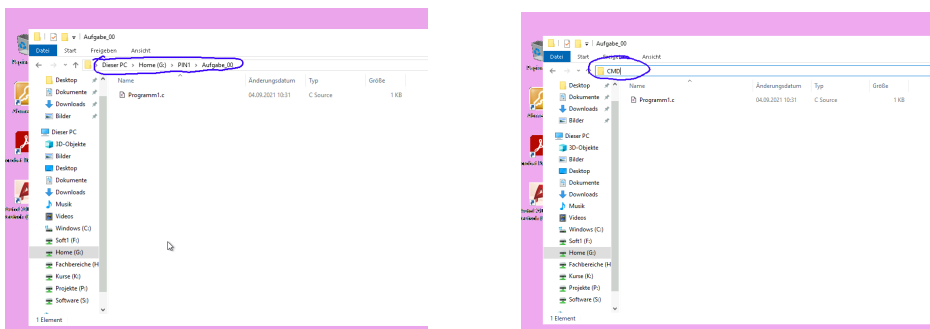


Abbildung 2: Die komfortabelste Methode die Textkonsole unter Windows zu starten: Klicken Sie im Verzeichnis, in dem Sie arbeiten wollen, oben in der Navigations-Adressleiste rechts neben den Verzeichnisnamen (siehe linkes Bild) und geben Sie dann den Namen des Konsolenprogramms CMD gefolgt von der Eingabetaste ein (siehe rechtes Bild). Dann wird die Konsole mit diesem Verzeichnis als „aktuellem Verzeichnis“ gestartet.

Linux: Hier gibt es eine bunte Vielfalt an sogenannten Terminal Emulatoren. Neben zahlreichen unabhängigen Terminals kommt auch jede Desktopoberfläche mit einer entsprechenden eigenen Anwendung. Verwenden Sie einfach, was Ihnen gefällt oder gerade installiert ist.

OS X: Hier existiert eine Standard-Anwendung `terminal`, die Sie verwenden können.

Windows: Auch hier gibt es eine Anwendung namens `CMD`, die Sie verwenden können. Sie können diese entweder im Startmenü Ihres Systems suchen und starten, oder aus dem Dateimanager wie in Abbildung 2 gezeigt, aufrufen.

Noch ein paar Worte zum Umgang mit der Konsole: Viele Dinge lassen sich mit der textbasierten Schnittstelle flexibler eleganter und schneller erledigen, als mit einer graphischen Schnittstelle.

Dabei hat die graphische Variante den Vorteil, dass man nicht alle Möglichkeiten kennen muss, sondern die Befehle in den Menüs suchen kann. Solange nicht zu viele Funktionen zu durchsuchen sind, ist das für Anfänger eine praktische Arbeitsweise

Mit zunehmender Erfahrung bzw. Komplexität der zu erledigenden Aufgaben wird die Maus aber oft zum bremsenden Element in der Arbeit.

Folgende Konzepte benötigen Sie im Umgang mit der Konsole:

Dateipfad: Auf Ihrem Rechner gibt es Verzeichnisse, die wiederum Verzeichnisse enthalten. Um unzweideutig eine Datei zu bezeichnen, muss man alle verschachtelten Verzeichnisse angeben, durch die man laufen muss (die man „aufklappen“ müsste) um zur Datei zu gelangen. Diesen „Weg“ zur Datei nennt man den Dateipfad. Einige Beispiele:

<code>/home/meins/PIN1/Aufgabe00/ersteDatei.c</code>	(Linux/OS X)
<code>~/PIN1/Aufgabe00/ersteDatei.c</code>	(Linux/OS X)
<code>G:PIN1/Aufgabe00/ersteDatei.c</code>	(Windows ³)
<code>C:0THfiles/PIN1/Aufgabe00/ersteDatei.c</code>	(Windows)

Aktuelles Verzeichnis: Jedes Fenster mit einer Textkonsole hat ein eigenes aktuelles Verzeichnis. Dieses kann durch das Programm `cd` (unter Windows) oder `pwd` (alle anderen) angezeigt werden.

³Windows verwendet aus historischen Gründen bevorzugt das Zeichen `\` für die Trennung der Elemente des Pfads. Allerdings kann auch in vielen Situationen das Zeichen `/` verwendet werden, wie es dem Standard bei anderen Systemen entspricht.

Im aktuellen Verzeichnis werden alle Dateien gesucht, für die man keinen expliziten Dateipfad angegeben hat. Das kann das Arbeiten natürlich enorm vereinfachen.

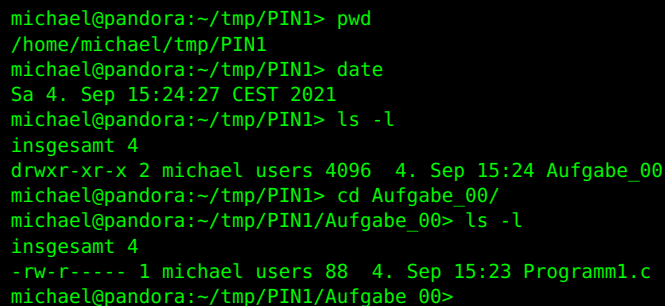
Verzeichniswechsel: Das aktuelle Verzeichnis kann man natürlich auch wechseln, dazu dient das Kommando `cd` dem man dazu das neue Verzeichnis als Parameter übergibt. Beispiele:

```
cd /home/meins/PIN1/Aufgabe00          (alle)
cd ~/PIN1/Aufgabe00                   (Linux/OS X)
cd ..                                  (alle, wechselt ins Elternverzeichnis)
```

Zu beachten ist hier nur, dass bei Windows für jedes Laufwerk ein eigenes aktuelles Verzeichnis gepflegt wird, das man durch `<Laufwerksbuchstabe>:` umschaltet und nicht mit `cd` wechselt:

G: (Windows)
C: (Windows)

Programm ausführen: Ein Programm führt man aus, indem man einfach seinen Namen (eventuell mit seinem Pfad) eingibt, und dann die Eingabetaste drückt (Beispiel für Linux/OS X):



```
michael@pandora:~/tmp/PIN1> pwd
/home/michael/tmp/PIN1
michael@pandora:~/tmp/PIN1> date
Sa 4. Sep 15:24:27 CEST 2021
michael@pandora:~/tmp/PIN1> ls -l
insgesamt 4
drwxr-xr-x 2 michael users 4096  4. Sep 15:24 Aufgabe_00
michael@pandora:~/tmp/PIN1> cd Aufgabe_00/
michael@pandora:~/tmp/PIN1/Aufgabe_00> ls -l
insgesamt 4
-rw-r----- 1 michael users 88  4. Sep 15:23 Programm1.c
michael@pandora:~/tmp/PIN1/Aufgabe_00>
```

Hier auch noch das analoge Beispiel für Windows, an dem zu erkennen ist, dass die Programme unter Windows etwas weniger kompakten Output erzeugen:

```
G:\PIN1>cd
G:\PIN1

G:\PIN1>date /T
04.09.2021

G:\PIN1>dir
Datenträger in Laufwerk G: ist home
Volumeseriennummer: 4A7C-AB0C

Verzeichnis von G:\PIN1

04.09.2021  10:18    <DIR>          .
04.09.2021  10:18    <DIR>          ..
04.09.2021  10:31    <DIR>          Aufgabe_00
                0 Datei(en),          0 Bytes
                3 Verzeichnis(se), 8.565.784.576 Bytes frei

G:\PIN1>cd Aufgabe_00

G:\PIN1\Aufgabe_00>dir
Datenträger in Laufwerk G: ist home
Volumeseriennummer: 4A7C-AB0C

Verzeichnis von G:\PIN1\Aufgabe_00

04.09.2021  10:31    <DIR>          .
04.09.2021  10:31    <DIR>          ..
04.09.2021  10:31                88 Programm1.c
                1 Datei(en),          88 Bytes
                2 Verzeichnis(se), 8.565.784.576 Bytes frei

G:\PIN1\Aufgabe_00>
```

Hinweis: Das sog. Kommandoprompt, das am Zeilenanfang ausgegeben wird, wenn von Ihnen ein neues Kommando erwartet wird, sieht übrigens bei jedem System anders aus und ist meist auch einstellbar.

Wichtig: Aus Gründen der Performance wird nicht der gesamte Rechner nach dem Programm durchsucht, sondern nur eine vordefinierte Liste von Verzeichnissen, der sogenannte Systempfad. Bei Programmen, die nicht in einem dieser besonderen Verzeichnisse liegen, muss das Programm mit seinem kompletten Pfad eingegeben werden, um ausgeführt werden zu können.

Programmparameter: Viele der Programme, die Sie verwenden werden, benötigen für Ihre Funktion zusätzliche Informationen (z.B: welche Datei eingelesen werden soll).

Diese Information wird über Parameter, die hinter dem Programmnamen angegeben und jeweils durch Leerzeichen voneinander getrennt werden, bereitgestellt.

Beispiele hierfür sind in den oben abgedruckten Sitzungsprotokollen (Screenshots) bereits enthalten.

Die Bedeutung, Reihenfolge und Schreibweise der Parameter ist dabei vom Programmierer des jeweiligen Programms individuell festgelegt worden und muss in der Programmanleitung nachgeschlagen werden.

4.3 Ein erster Versuch

???

Öffnen Sie ein Konsolenfenster.

Öffnen Sie nun ein Konsolenfenster und versuchen Sie die Anwendung gcc zu starten, indem Sie die drei Buchstaben in das Konsolenfenster eingeben und über die Eingabetaste starten. Wenn alles wie gewünscht geklappt hat, sollte das Programm starten und sich beklagen, dass Eingabedaten fehlen. Das könnte zum Beispiel so aussehen:

```
Programme/Erstes> gcc
gcc: schwerwiegender Fehler: keine Eingabedateien
Kompilierung beendet.
```

Je nach Systemkonfiguration wird obiger Test fehlschlagen, weil das Verzeichnis, in dem der Compiler installiert wurde, nicht im Systemsuchpfad enthalten ist und das Programm gcc daher nicht gefunden wird.

Linux: Hier sollte das eigentlich klappen.

Windows: Suchen Sie den Installationsort Ihres Compilers auf Ihrem Rechner. Die Datei heißt gcc.exe. Typischerweise werden Sie hier fündig:

```
c:\Program Files\CodeBlocks\MinGW\bin
```

– aber der Pfad kann durchaus auch systemabhängig anders lauten. Wenn Sie die ausführbare Datei Ihres Compilers gefunden haben, geben Sie folgendes Kommando in Ihrer Eingabeaufforderung ein:

```
PATH %PATH%;c:\Program Files\CodeBlocks\MinGW\bin
```

aber achten Sie dabei den korrekten Pfad einzusetzen, den Sie auf Ihrem System ermittelt haben. Der Pfad darf *nicht* in Anführungszeichen stehen.

Wichtig: Dieses Kommando müssen Sie jedes Mal neu eingeben, wenn Sie ein neues CMD-Fenster öffnen.⁴

4.4 Ein erster Erfolg



Übersetzen Sie Ihr erstes Programm und führen Sie es aus.

Nun können Sie Ihr erstes Programm übersetzen. Führen Sie hierfür folgendes Kommando aus:

```
gcc MeinErstesProgramm.c
```

wobei Sie natürlich den von Ihnen gewählten Programmnamen für Ihr Programm verwenden. Wenn alles geklappt hat, erhalten

⁴Um den Systempfad dauerhaft zu erweitern, müssen Sie je nach Windows-Version etwas anders vorgehen. Versuchen Sie im Start-Menü nach „Environment“ zu suchen und dann einen Dialog zum Bearbeiten des „Environments“ oder der „Umgebung“ zu suchen. Dort finden Sie dann eine Variable namens „PATH“ deren Inhalt sie wie im Text beschrieben um den Pfad zum gcc.exe ergänzen.

Sie eine Datei `a.exe` (Windows) bzw. `a.out` (andere Systeme), die das Betriebssystem dem Prozessor nun zur direkten Ausführung übergeben kann. Man nennt diese Datei daher auch „das Executable“.

Führen Sie nun diese Datei aus.

Windows: Geben Sie hierfür `a` als Programmname an der Konsole an.

andere Systeme: Geben Sie hierfür `./a.out` als Programmname an der Konsole an.

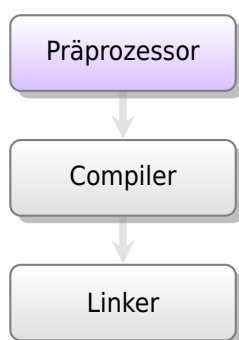
Hinweis: Sie können die Datei natürlich auch starten, indem Sie sie in der graphischen Oberfläche doppelt anklicken. Allerdings werden Sie dann nur ein kurzes Aufblitzen wahrnehmen, da das Programm nur ganz kurz läuft und dann das Fenster gleich wieder geschlossen wird.

Natürlich ist der Name des erzeugten Programms noch etwas unbefriedigend. Daher löschen Sie das Executable wieder und modifizieren Sie Ihren Aufruf des Compilers, indem Sie den Wunschnamen für die Ausgabedatei mit angeben:

```
gcc -o MeinErstesProgramm MeinErstesProgramm.c
```

Hinweis: Damit Windows die ausführbare Datei erkennt, muss sie die Dateiendung `.exe` besitzen. Diese wird automatisch angehängt. Bei Linux-Systemen wird entsprechend das „execute“-Bit in den Dateieigenschaften gesetzt.

4.5 Der Präprozessor



Der Übersetzungsvorgang eines C-Programms läuft in mehreren Phasen ab, welche der gcc-Compiler im letzten Experiment praktischerweise zu einem Aufruf zusammengefasst hat. Dies ist aber nicht in jedem Fall so erwünscht.

In der Vorlesung lernen Sie *später* die verschiedenen Phasen, wie z.B. den Präprozessor, kennen und auch, wie man diesen steuert und für die eigene Zwecke sinnvoll einsetzt. Der Präprozessor erlaubt es nämlich, am Programmcode automatisch einfache Veränderungen wie das Einfügen anderer Dateien oder das Ersetzen von Textteilen vorzunehmen.

Wird der Compiler mit der Option `-E` aufgerufen, so bricht er nach der Präprozessorstufe ab und liefert das entsprechende Zwischenergebnis. So kann der Effekt des Präprozessors auf



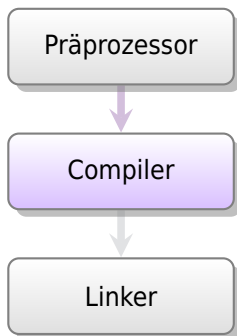
Lassen Sie sich den präprozessierten Quelltext Ihres Programms ausgeben.

Korrektheit untersucht werden:

```
gcc -o MeinErstesProgramm_prepro.txt -E MeinErstesProgramm.c
```

Frage: Sehen Sie sich den präprozessierten Quelltext Ihres Programms an. Können Sie darin Ihren Code wiedererkennen? Wie erklären Sie sich das? Diskutieren Sie das Thema mit Ihren Dozenten.

4.6 Der Compiler



Eine weitere Stufe der Übersetzung des Programms in eine vom Prozessor ausführbare Zahlenfolge ist die sogenannte Assemblersprache. Hier werden die Befehle des Prozessors noch als menschenlesbare Textkürzel angezeigt.

Sie können den Compiler über die Angabe der Option `-S` dazu bringen, den Übersetzungsvorgang an dieser Stelle abubrechen und den Assemblercode auszugeben. Damit haben Sie die Möglichkeit, den Assemblercode zu analysieren, um Fehler des Compilers zu finden oder Laufzeitanalysen durchzuführen⁵.

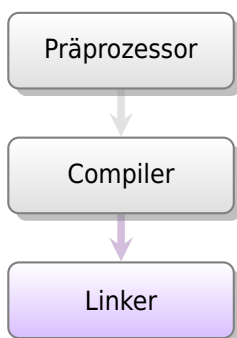
```
gcc -S MeinErstesProgramm.c
```

Frage: Sehen Sie sich den Assemblercode Ihres Programms an. Natürlich sprechen Sie die Sprache nicht. Aber was ist Ihr Eindruck?



Lassen Sie sich den Assembler-Quellcode Ihres Programms ausgeben.

4.7 Mehrteilige Programme



Will man ernstzunehmende Probleme lösen, so wird ein Programm, das nur in einer Datei steht, schnell unübersichtlich und praktisch schwer zu handhaben. Daher unterstützen alle Programmiersprachen Konzepte, ein Programm auf mehrere Dateien aufzuteilen. Neben der leichteren Handhabbarkeit (weil beim Editieren das Herumnavigieren im Programm über weite Strecken entfällt) ergibt



Erzeugen Sie ein mehrteiliges Softwareprojekt und übersetzen Sie es.

⁵Allerdings erfordert dies gute Kenntnisse der jeweiligen Assemblersprache sowie dem Aufbau des verwendeten Prozessors. Entsprechende Einblicke erhalten Sie je nach Studiengang in späteren Semestern.

sich daraus die Möglichkeit, solche Teile auch in mehreren Programmen wiederzuverwenden und beim Übersetzen eines Programms auch nur diejenigen Teile neu zu übersetzen, welche auch tatsächlich von Änderungen betroffen sind.

Auch in diesem Praktikum PIN1 werden Sie bereits mit solchen mehrteiligen Programmen arbeiten.

Hier benötigen wir zwei Programmteile. Erzeugen Sie also (am Besten in einem neuen Verzeichnis) drei neue Dateien, nämlich `maincode.c` mit dem Inhalt:

```
#include <stdio.h>
#include "libcode.h"
int main(int argc, char** argv)
{
    printf("Das Ergebnis ist %i\n", myOwnFunc(42,12));
}
```

und `libcode.h` mit dem Inhalt:

```
extern int myOwnFunc(int x, int y);
```

und `libcode.c` mit dem Inhalt:

```
#include "libcode.h"
int myOwnFunc(int x, int y)
{
    return x*y+2*x-y;
}
```

Hinweis: Falls Sie sich fragen, wozu die zusätzliche `.h`-Datei dient: Sie enthält Informationen, die vom Compiler beim Übersetzen *beider* `.c`-Dateien benötigt werden und die somit nur einmal getippt werden müssen.

Nun können Sie versuchen, die beiden `.c`-Dateien zu übersetzen, was aber zu Problemen führt⁶.

Die Ursache ist in beiden Fällen, dass der jeweils andere Teil zu einem vollständigen Programm fehlt. Die Lösung besteht nun darin, beide Teile gewissermaßen als „Halbzeuge“ vorzufertigen (also insbesondere auch die Assemblerkommandos in Zahlen zu übersetzen, die dann der Prozessor auch verarbeiten kann).

```
gcc -c libcode.c
gcc -c maincode.c
```

Diese beiden Läufe erzeugen die Dateien `libcode.o` und `maincode.o` welche die fertig compilierten „Halbzeuge“ darstellen.

⁶Sie können das Problem lösen, indem Sie dem Compiler beide `.c`-Dateien als Parameter mitgeben. Diese Möglichkeit wollen wir hier aber nicht betrachten.

Diese Dateien sind binär, enthalten also keine im Editor anzeigbaren Zeichen, sondern nur Zahlen. Weil das ein wenig unübersichtlich zu lesen ist, existiert das Programm `objdump` mit dessen Hilfe sich der Inhalt dieser Objektdateien in übersichtlicher Weise darstellen lässt.

Lassen Sie sich die generierten Dateien über den Aufruf von
`objdump -s <hierkommtderfilenamehin>`
anzeigen. Sie erhalten eine Ausgabe der Form:

```
libcode.o:      file format pe-x86-64

Contents of section .text:
 0000 554889e5 894d1089 55188b45 100faf45  UH...M..U..E...E
 0010 188b5510 01d201d0 2b45185d c3909090  ..U.....+E.]....
Contents of section .xdata:
 0000 01040205 04030150                .....P
Contents of section .pdata:
 0000 00000000 1d000000 00000000                .....
Contents of section .rdata$zzz:
 0000 4743433a 20287838 365f3634 2d706f73  GCC: (x86_64-pos
 0010 69782d73 65682d72 6576302c 20427569  ix-seh-rev0, Bui
 0020 6c742062 79204d69 6e47572d 57363420  lt by MinGW-W64
 0030 70726f6a 65637429 20382e31 2e300000  project) 8.1.0..
```

Links sehen Sie zur Orientierung die Nummer der entsprechenden Datenbytes, im mittleren Bereich die Werte der Zahlen als Hexadezimalwerte (näheres dazu lernen Sie später in der Vorlesung) und im rechten Bereich die zu den Zahlen gehörigen Zeichen laut ASCII-Kodierung (auch das hören Sie später in der Vorlesung).

Weil das jetzt etwas schwierig zu entziffern ist, kann `objdump` die Zahlen der Befehle auch wieder in Assemblercode zurückübersetzen („disassemblieren“). Das Erreichen Sie über die Option
`objdump -S`

und erhalten ein Ergebnis so ähnlich wie dies hier:

```
libcode.o:      file format pe-x86-64

Disassembly of section .text:
0000000000000000 <myOwnFunc>:
 0: 55          push    %rbp
 1: 48 89 e5    mov     %rsp,%rbp
 4: 89 4d 10    mov     %ecx,0x10(%rbp)
 7: 89 55 18    mov     %edx,0x18(%rbp)
 a: 8b 45 10    mov     0x10(%rbp),%eax
 d: 0f af 45 18 imul    0x18(%rbp),%eax
11: 8b 55 10    mov     0x10(%rbp),%edx
14: 01 d2      add     %edx,%edx
16: 01 d0      add     %edx,%eax
18: 2b 45 18    sub     0x18(%rbp),%eax
1b: 5d          pop     %rbp
1c: c3          retq
1d: 90          nop
1e: 90          nop
1f: 90          nop
```

Erst zum Schluss, wenn alle diese Halbzeuge fertig vorliegen, werden sie in einem weiteren Lauf zu einem kompletten Programm zusammengefügt – das ist dann die sogenannte „link&locate“-Phase, in der alle Referenzen auf andere Programmteile aufgelöst werden:

```
gcc -o maincode libcode.o maincode.o
```

Dabei entsteht nun eine ausführbare Datei (unter Windows: .exe), die Sie ebenfalls mit objdump untersuchen können.

Hinweis: Es wird Ihnen auffallen, dass das Programm viel größer ist, als die beiden o.-Dateien zusammen. Dies kommt daher, dass der Compiler viele Programmteile in vorkompilierter Form (also Halbzeuge wie unsere Objekt-Dateien) mitbringt und diese mit zu dem Programm hinzufügt. Dies betrifft zum Beispiel die Implementierung von `printf` und viele andere Dinge, die zum Beispiel zur Kommunikation mit dem Betriebssystem benötigt werden.

5 Automatisierung des Buildprozesses

Damit haben Sie nun ihre ersten beiden C-Programme geschrieben und mit einem Compiler übersetzt. Das ist natürlich so ziemlich umständlich. Daher gibt es zahlreiche Anwendungen, die diesen Vorgang erleichtern.

Allen voran ist hier `make` zu nennen. Dieses Programm wird über ein sogenanntes Makefile gesteuert, dessen Aufbau zu erklären den Rahmen hier sprengen würde.

Schreiben Sie folgenden Dateiinhalt in eine Datei namens Makefile:

```
CC=gcc
BINARY=maincode.exe

$(BINARY): maincode.o libcode.o
    $(CC) -o $@ maincode.o libcode.o

maincode.o: maincode.c libcode.h
    $(CC) -c -o $@ $<

libcode.o: libcode.c libcode.h
    $(CC) -c -o $@ $<
```

Wichtig: Die Datei *muss* den Namen Makefile oder makefile haben. Manche Editoren hängen beim Speichern Suffixe wie `.txt` an. Dann wird das Makefile nicht mehr gefunden. Windows unterdrückt die Endungen je nach den Explo-



Übersetzen Sie Ihr Softwareprojekt mittels `make`.

rer-Ansichtseinstellungen – was die Lage nicht übersichtlicher macht⁷.

Am Besten benennen Sie die Datei nach dem Speichern in Makefile um, indem Sie eventuelle Anhängsel am Namen entfernen⁸.

Wichtig: Achten Sie darauf, die eingerückten Zeilen *unbedingt* mit einem Tabulatorzeichen einzurücken (das Tabulatorzeichen finden Sie auf der Taste oberhalb der Großschreib-Feststelltaste). Sonst wird das Makefile nicht funktionieren.

Wichtig: Linux Anwender sollten hier im Makefile den Namen des Binarys korrigieren, indem sie die Endung entfernen.

Wenn Sie das Makefile erzeugt haben, reicht ein einfacher Aufruf von `mingw32-make` (bei Windows) bzw. `make` (bei Linux) und alle Schritte laufen automatisch ab:

```
gcc -c -o maincode.o maincode.c
gcc -c -o libcode.o libcode.c
gcc -o maincode.exe maincode.o libcode.o
```

Hinweis: Löschen Sie vor dem Test des Makefiles bitte das Binary und die beiden `.o`-Dateien. `make` ist wie Softwareentwickler: faul. Es arbeitet nur, wenn Bedarf besteht.

Das Generieren von Makefiles für komplexere Projekte wird unterstützt von Lösungen wie `cmake`.

Natürlich existieren auch die diversen integrierten Entwicklungsumgebungen (IDE), welche nicht nur den Übersetzungsvorgang automatisieren, sondern auch andere praktische Funktionen sowie einen Editor für den Quellcode integrieren. Es ist allerdings für deren Verständnis wichtig, sich immer zu vergegenwärtigen, dass im Hintergrund genau die selben Schritte ablaufen, wie Sie sie hier in dieser Aufgabe kennengelernt haben – auch mit den selben Werkzeugen.

Ein Beispiel für so eine Entwicklungsumgebung (das bereits installierte CodeBlocks) lernen Sie in der nächsten Aufgabe kennen.

⁷Tipp: Schalten Sie unbedingt das Unterdrücken der Dateiendungen im Explorer ab. Das schadet viel mehr als es nützt.

⁸Alternativ können Sie beim Aufruf des `make`-Programms auch den tatsächlichen Namen der Datei mit der Option `-f` beim Aufruf von `make` angeben. Das ist aber umständlich.